

Loyola University Chicago
Department of Computer Science

COMP 271: Data Structures I
Quiz 1 Practice Exam (Weeks 1–4)
Suggested time: 50 minutes

September 25, 2026

Full Name: _____



This is a practice exam. It is not graded. It covers the same material as Quiz 1 (Weeks 1 to 4: refactoring, type annotations, classes and special methods, encapsulation and properties, inheritance, polymorphism, abstraction, and the Programmer’s Pact), and it is about twice as long.

To get the most out of it, work under real conditions: closed book, no notes, no devices, and a timer set to 50 minutes. Then check your answers against the solutions and review the topics you missed.

For coding questions, write Python that follows our course style (the Programmer’s Pact): type hints, a one-line docstring, named constants instead of magic values, no `break/continue`, and one `return` per function.

1. **Refactoring.** Which statement best describes the goal of refactoring?

- ☐ Adding new features to a program as quickly as possible.
- ☒ **Improving the structure of the code (less duplication, clearer names, behavior driven by data) while keeping its output the same.**
- ☐ Rewriting a program in fewer lines, even if the output changes slightly.
- ☐ Replacing every loop with a separate function.

Solution: Refactoring changes *how* the code is written, never *what* it does. Every version of “The House That Jack Built” printed the same rhyme.

2. **Dynamic vs. strong typing.** Consider:

```
1 x = 5
2 x = "five"
3 y = "3" + 4
```

Which statement is true?

- ☐ Line 2 raises an error, because `x` was already an `int`.
- ☐ Line 3 sets `y` to `"34"`.
- ☐ Line 3 sets `y` to 7.
- ☒ **Line 2 is fine (Python is dynamically typed), but line 3 raises a `TypeError` (Python is strongly typed).**

Solution: Dynamic typing: a name can be rebound to a value of another type. Strong typing: Python does not silently convert `"3"` and `4` to a common type.

3. Type annotations. What happens when this runs?

```
1 def square(n: int) -> int:
2     """Return n times n."""
3     return n * n
4
5 print(square(2.5))
```

- ☐ A `TypeError`, because 2.5 is not an `int`.
- ☐ It prints 6, because the result is converted to `int`.
- ☒ It prints 6.25; annotations are hints and are not enforced when the program runs.
- ☐ A `SyntaxError`, because annotations cannot be used with floats.

Solution: Annotations document intent and can be checked by a separate tool such as `mypy`, but Python itself ignores them at runtime.

4. Class variables vs. instance variables.

```
1 class Robot:
2     _count = 0
3
4     def __init__(self, name: str) -> None:
5         self._name = name
6         Robot._count += 1
7
8 Robot("Ada")
9 Robot("Bolt")
10 Robot("Cog")
11 print(Robot._count)
```

(a) What is printed?

- ☐ 0
- ☐ 1
- ☒ 3
- ☐ An error, because `_count` is private.

(b) Suppose line 6 is changed to `self._count += 1`. What is printed now?

- ☒ 0
- ☐ 1
- ☐ 3
- ☐ An error, because `self` has no `_count`.

Solution: (a) `_count` is a class variable shared by all robots, so three objects give 3. (b) `self._count += 1` reads the class value (0) but *writes* a new instance variable on each object, so `Robot._count` stays 0.

5. `__str__` and `__repr__`.

```
1 class Point:
2     def __init__(self, x: int, y: int) -> None:
3         self.__x = x
4         self.__y = y
5
6     def __str__(self) -> str:
7         return f"({self.__x}, {self.__y})"
8
9 p = Point(1, 2)
10 print(p)
11 print([p])
```

What do the two `print` calls show?

- ☐ (1, 2) and [(1, 2)]
- ☒ (1, 2) and something like `<__main__.Point object at 0x...>`
- ☐ Both show something like `<__main__.Point object at 0x...>`
- ☐ An error, because `__repr__` is missing.

Solution: `print(p)` uses `__str__`, but a list displays its items with `__repr__`, which was not defined. That is why we often write `__repr__` to return `str(self)`.

6. `__eq__`. `Point` now has this method (and `key()` returns `(x, y)`):

```
1 def __eq__(self, other: object) -> bool:
2     """Compare two points by their coordinates."""
3     result = NotImplemented
4     if isinstance(other, Point):
5         result = self.key() == other.key()
6     return result
```

What does `Point(1, 2) == "hello"` evaluate to?

- ☐ An error, because `NotImplemented` is not a boolean.
- ☐ `NotImplemented`
- ☒ **False**
- ☐ `True`

Solution: Returning `NotImplemented` tells Python “I don’t know how to compare with this type”. Python then tries the other side, finds nothing, and falls back to **False**.

7. **Sorting objects.** `songs` is a `list[Song]`. Which special method must `Song` define so that `sorted(songs)` works?

- ☐ `__eq__`
- ☒ **`__lt__`**
- ☐ `__len__`
- ☐ `__str__`

Solution: Sorting compares items with `<`, which calls `__lt__`.

8. **UML relationships.** A **Team** keeps a list of **Player** objects. Players exist on their own: a player can leave one team and join another, and deleting a team does not delete its players. Which UML relationship fits best?

- ☐ Inheritance
- ☒ **Aggregation**
- ☐ Composition
- ☐ Realization

Solution: Aggregation is a “has-a” whose parts can live independently. With composition, the parts would be destroyed with the whole.

9. **Method resolution order.**

```
1 class A:
2     def greet(self) -> str:
3         return "A"
4
5 class B(A):
6     def greet(self) -> str:
7         return "B"
8
9 class C(B):
10     pass
```

What is `C.mro()`, and what does `C().greet()` return?

- ☐ [A, B, C, object], and "A"
- ☒ **[C, B, A, object], and "B"**
- ☐ [C, A, B, object], and "A"
- ☐ [C, object], and an error because C has no `greet`

Solution: Python searches the class itself first, then its parents in order. C has no `greet`, so it uses the first one found: `B.greet`.

10. **The Programmer's Pact.** Both functions return True when target is in items. Which one follows the Pact?

Version 1

```

1 def has(items: list[str],
2     target: str) -> bool:
3     """Say if target is in items."""
4     for item in items:
5         if item == target:
6             return True
7     return False

```

Version 2

```

1 def has(items: list[str],
2     target: str) -> bool:
3     """Say if target is in items."""
4     found = False
5     i = 0
6     while i < len(items) and not found:
7         found = items[i] == target
8         i += 1
9     return found

```

- ☐ Version 1, because it is shorter.
- ☒ **Version 2: the stopping condition is in the loop condition and there is a single return.**
- ☐ Both, since they return the same result.
- ☐ Neither, because both use a loop.

Solution: Version 1 has two `return` statements and exits the loop early from inside it. Version 2 puts the stopping condition in the `while` condition and returns once.

11. **Single underscore.**

```

1 class Pet:
2     def __init__(self, name: str) -> None:
3         self._name = name
4
5 print(Pet("Rex")._name)

```

Which statement is true?

- ☒ **It prints Rex. A single underscore only signals “internal use”; Python does not block access.**
- ☐ It raises an `AttributeError`, because `_name` is private.
- ☐ It prints `None`.
- ☐ A single underscore makes `_name` a class variable.

Solution: A leading underscore is a convention between programmers (“we are all adults”). Nothing stops outside code from reading it; it just shouldn’t.

12. Abstract base classes.

```
1 from abc import ABC, abstractmethod
2
3 class Report(ABC):
4     def header(self) -> str:
5         return "== Report =="
6
7     @abstractmethod
8     def render(self) -> str:
9         """Return the full report text."""
10
11 class SalesReport(Report):
12     def render(self) -> str:
13         return self.header() + " sales"
```

Which statement is true?

- ☐ An abstract class cannot contain any concrete method such as `header()`.
- ☐ `SalesReport` must also redefine `header()`.
- ☒ `SalesReport().render()` returns `"== Report == sales"`: the subclass inherits the concrete `header()`.
- ☐ `header()` runs automatically when a `SalesReport` is created.

Solution: An abstract base class can mix shared, concrete behavior (`header`) with abstract methods that each subclass must supply (`render`).

13. Aliasing. What is printed?

```
1 a = [1, 2]
2 b = a
3 b.append(3)
4 print(a)
```

- ☐ `[1, 2]`
- ☒ `[1, 2, 3]`
- ☐ `[3]`
- ☐ An error

Solution: `b = a` does not copy the list; both names refer to the same list object.

14. **None vs. an empty value.** A function looks up a user's email:

```
1 def find_email(username: str) -> str | None:
```

Why is returning `None` when the user does not exist better than returning `""`? How should the caller check the result?

Solution: `None` means “no value at all”, which is different from a real but empty string (a user could exist with an empty email field). The annotation `str | None` also tells callers they must handle the missing case. The caller checks `if result is None:` (or `is not None`) before using the email.

15. **Properties vs. getter/setter methods.** A class `Account` currently has methods `get_owner()` and `set_owner(new_owner)`, used like this:

```
1 print(acct.get_owner())  
2 acct.set_owner("Maya")
```

- (a) Rewrite the two lines as they would look if `owner` were a `@property` with a setter.

Solution: `print(acct.owner)` and `acct.owner = "Maya"`

- (b) Give one advantage of the `@property` style and one advantage of explicit `get_/set_` methods.

Solution: `@property`: idiomatic Python, reads like a plain attribute, and validation can be added later without changing any calling code. `get_/set_`: explicit about what happens, and the same style carries over to languages such as Java.

16. **Following the Pact.** List three ways this function breaks the Programmer's Pact, then rewrite it so it follows the Pact.

```
1 def shipping(weight):  
2     if weight > 20:  
3         return weight * 1.5 + 10  
4     return weight * 1.5
```

Solution: Missing type hints; missing docstring; magic values (20, 1.5, 10); two `return` statements. One possible rewrite:

```
1 HEAVY_LIMIT_KG = 20  
2 RATE_PER_KG = 1.5  
3 HEAVY_SURCHARGE = 10  
4  
5 def shipping(weight: float) -> float:  
6     """Return the shipping cost for a package of the given weight."""  
7     cost = weight * RATE_PER_KG  
8     if weight > HEAVY_LIMIT_KG:  
9         cost += HEAVY_SURCHARGE  
10    return cost
```

17. Reading a UML class diagram.

BankAccount
- __owner: str
- __balance: float
+ deposit(amount: float): None
+ balance(): float

- (a) What do the - and + signs mean?

Solution: - means private, + means public.

- (b) Write the Python skeleton of this class: the class header, an `__init__` that sets both attributes, and the two method headers with type hints (bodies can be ...).

Solution:

```
1 class BankAccount:
2     """A bank account with an owner and a balance."""
3
4     def __init__(self, owner: str, balance: float) -> None:
5         """Create an account."""
6         self.__owner = owner
7         self.__balance = balance
8
9     def deposit(self, amount: float) -> None:
10        """Add amount to the balance."""
11        ...
12
13    def balance(self) -> float:
14        """Return the current balance."""
15        ...
```

18. **Tracing `super()`.** What does this program print?

```
1 class Base:
2     def __init__(self) -> None:
3         print("Base init")
4
5 class Middle(Base):
6     def __init__(self) -> None:
7         print("Middle start")
8         super().__init__()
9         print("Middle end")
10
11 class Leaf(Middle):
12     def __init__(self) -> None:
13         super().__init__()
14         print("Leaf init")
15
16 Leaf()
```

Solution: Middle start, Base init, Middle end, Leaf init (one per line). `Leaf.__init__` calls `Middle.__init__` first, which calls `Base.__init__` in the middle of its own work.

19. **Refactor with one helper.** These three functions are almost identical:

```
1 def total_even(numbers: list[int]) -> int:
2     total = 0
3     for n in numbers:
4         if n % 2 == 0:
5             total += n
6     return total
7
8 def total_odd(numbers: list[int]) -> int:
9     total = 0
10    for n in numbers:
11        if n % 2 == 1:
12            total += n
13    return total
14
15 def total_multiples_of_three(numbers: list[int]) -> int:
16     total = 0
17     for n in numbers:
18         if n % 3 == 0:
19             total += n
20    return total
```

- (a) Write one helper `total_matching(numbers: list[int], divisor: int, remainder: int) -> int` that returns the sum of the numbers `n` for which `n % divisor == remainder`.

Solution:

```
1 def total_matching(numbers: list[int], divisor: int, remainder: int) -> int:
2     """Return the sum of the numbers leaving remainder when divided by
3     divisor."""
4     total = 0
5     for n in numbers:
6         if n % divisor == remainder:
7             total += n
8     return total
```

- (b) Rewrite `total_odd` as a one-line wrapper around your helper.

Solution: `return total_matching(numbers, 2, 1)` (with a docstring; naming the 2 as a constant such as `EVEN_ODD_DIVISOR` is even better).

- (c) Explain why adding `total_multiples_of_five` is now much cheaper than before.

Solution: All the logic lives in one place; a new variant is one call with different arguments (`total_matching(numbers, 5, 0)`), not another copy of the loop. A bug fix in the helper also fixes every wrapper.

20. **A class with special methods.** Write a class `Song` that:

- (a) Stores a `title`, an `artist` and a length in `seconds` (`int`) in **private** attributes set by `__init__`, and exposes each one through a read-only `@property`.
- (b) Defines `__str__` so that `print(Song("Echo", "Lumen", 185))` shows `Echo by Lumen (185s)`, and `__repr__` so that a list of songs prints the same way.
- (c) Defines `__eq__` so that two songs are equal when they have the same title and artist (the length does not matter), using a `key()` method, and returns `NotImplemented` when compared with something that is not a `Song`.
- (d) Defines `__lt__` so that `sorted(songs)` orders songs from shortest to longest.

Solution:

```

1  class Song:
2      """A song with a title, an artist and a length in seconds."""
3
4      def __init__(self, title: str, artist: str, seconds: int) -> None:
5          """Create a song."""
6          self.__title = title
7          self.__artist = artist
8          self.__seconds = seconds
9
10     @property
11     def title(self) -> str:
12         """Return the title."""
13         return self.__title
14
15     @property
16     def artist(self) -> str:
17         """Return the artist."""
18         return self.__artist
19
20     @property
21     def seconds(self) -> int:
22         """Return the length in seconds."""
23         return self.__seconds
24
25     def key(self) -> tuple[str, str]:
26         """Return the fields that identify a song."""
27         return (self.__title, self.__artist)
28
29     def __str__(self) -> str:
30         """Return a readable description."""
31         return f"{self.__title} by {self.__artist} ({self.__seconds}s)"
32
33     def __repr__(self) -> str:
34         """Use the same text as __str__."""
35         return str(self)
36
37     def __eq__(self, other: object) -> bool:
38         """Two songs are equal when title and artist match."""
39         result = NotImplemented

```

```
40     if isinstance(other, Song):
41         result = self.key() == other.key()
42     return result
43
44     def __lt__(self, other: "Song") -> bool:
45         """Order songs from shortest to longest."""
46         return self.__seconds < other.seconds
```

21. **Abstraction, inheritance and polymorphism.** You are given:

```

1 from abc import ABC, abstractmethod
2
3 class PhonePlan(ABC):
4     """A phone plan with a name and a base price."""
5     _plans_created = 0
6
7     def __init__(self, name: str, base_price: float) -> None:
8         """Create a plan and count it."""
9         self.__name = name
10        self.__base_price = base_price
11        PhonePlan._plans_created += 1
12
13    @property
14    def name(self) -> str:
15        """Return the plan name."""
16        return self.__name
17
18    @property
19    def base_price(self) -> float:
20        """Return the base price."""
21        return self.__base_price
22
23    def describe(self) -> str:
24        """Return a one-line summary of the plan."""
25        return f"{self.name}: ${self.monthly_cost()} per month"
26
27    @abstractmethod
28    def monthly_cost(self) -> float:
29        """Return what the plan costs each month."""

```

(a) Write `BasicPlan(PhonePlan)`, whose monthly cost is just the base price.

Solution:

```

1 class BasicPlan(PhonePlan):
2     """A plan that costs only its base price."""
3
4     def monthly_cost(self) -> float:
5         """Return the base price."""
6         return self.base_price

```

(b) Write `UnlimitedPlan(PhonePlan)`. Its constructor also takes a `data_fee`, reuses the parent's constructor for the name and base price, and stores the fee in a private attribute. Its monthly cost is the base price plus the data fee.

Solution:

```
1 class UnlimitedPlan(PhonePlan):
2     """A plan with a data fee on top of the base price."""
3
4     def __init__(self, name: str, base_price: float, data_fee: float) ->
5     None:
6         """Create an unlimited plan."""
7         super().__init__(name, base_price)
8         self.__data_fee = data_fee
9
10    def monthly_cost(self) -> float:
11        """Return the base price plus the data fee."""
12        return self.base_price + self.__data_fee
```

Using `self.__base_price` here would fail: inside `UnlimitedPlan` it is renamed to `_UnlimitedPlan__base_price`. Use the property.

- (c) Write `cheapest_plan(plans: list[PhonePlan]) -> PhonePlan | None` that returns the plan with the lowest monthly cost, or `None` if the list is empty. Do not check the type of any plan, and use a single `return`.

Solution:

```
1 def cheapest_plan(plans: list[PhonePlan]) -> PhonePlan | None:
2     """Return the cheapest plan, or None if there are no plans."""
3     cheapest = None
4     for plan in plans:
5         if cheapest is None or plan.monthly_cost() < cheapest.monthly_cost():
6             cheapest = plan
7     return cheapest
```

- (d) After running `BasicPlan("Basic", 30.0)` and `UnlimitedPlan("Max", 25.0, 20.0)`, what is `PhonePlan._plans_`? Why?

Solution: 2. It is a class variable shared by every plan, and both constructors run `PhonePlan.__init__` (directly or through `super()`), which adds 1.

- (e) `describe()` is written once, in `PhonePlan`, yet it shows `Basic: $30.0 per month` for the first plan and `Max: $45.0 per month` for the second. Explain what makes this work.

Solution: Inheritance: both subclasses inherit `describe()`. Polymorphism: inside it, `self.monthly_cost()` runs the version belonging to the actual object's class (found through the MRO), so each plan computes its own cost with no type checks.